Estado del arte y reproducción de LectureKick para CB-CTT

Alejandro
Máster Universitario en Aprendizaje Automático y Datos Masivos
Universidad Politécnica de Madrid

Resumen. **Este trabajo revisa las principales metaheurísticas, métodos híbridos y técnicas de aprendizaje automático aplicadas al problema de planificación universitaria basada en currículos. El estudio se centra después en LectureKick, un vecindario que amplía el movimiento LectureMove al permitir que una clase desplazada se reasigne a un tercer hueco. El operador se integra en un esquema de recocido simulado con sesgos dirigidos a las restricciones blandas de ITC-2007. Finalmente se define una reproducción trazable del código de Ceschia et al. sobre 19 instancias, tres semillas por instancia y un presupuesto calibrado de 192 segundos. Se registran versiones, comandos, soluciones y tiempos para facilitar la comparación posterior.**

Index Terms. **CB-CTT, planificación universitaria, metaheurísticas, LectureKick, recocido simulado, reproducibilidad**

I. Metaheurísticas, híbridos y aprendizaje

La planificación universitaria basada en currículos, conocida como CB-CTT, mantiene un espacio de búsqueda amplio incluso cuando la factibilidad se alcanza con rapidez. Esta diferencia explica el peso de las metaheurísticas en la literatura. Los métodos exactos pueden demostrar optimalidad o mejorar cotas con presupuestos largos, mientras que la búsqueda local concentra el tiempo disponible en obtener horarios de buena calidad. La comparación solo resulta informativa cuando la formulación, las instancias y el límite de ejecución coinciden [1].

I-A. Metaheurísticas

Los primeros trabajos sobre CB-CTT combinaron movimientos de reasignación e intercambio con Hill Climbing y búsqueda tabú. El ganador de la pista curricular de ITC-2007 amplió ese planteamiento mediante cinco vecindarios coordinados con Hill Climbing, Great Deluge y recocido simulado [2]. Su fortaleza no procedía de un único operador, sino de alternar fases de intensificación y escape sin abandonar la factibilidad.

El recocido simulado acepta mejoras y admite ciertos empeoramientos según la temperatura. Esa tolerancia evita que la búsqueda quede detenida en el primer óptimo local. Bellio et al. mostraron que un esquema sencillo podía ser competitivo si sus parámetros se relacionaban con características de cada instancia [3]. Song et al. siguieron otra vía. Su búsqueda local con varios vecindarios ajusta las probabilidades de selección y reinicia desde la mejor de dos trayectorias de recocido [4]. Ambos estudios muestran que la elección del movimiento y su frecuencia importan tanto como la regla de aceptación.

VNS y VND cambian de vecindario de forma sistemática cuando una estructura deja de producir mejoras. Las hiperheurísticas trasladan esa decisión a un nivel superior y escogen entre operadores de bajo nivel. Estos enfoques reducen la dependencia de un movimiento concreto, aunque incorporan nuevas decisiones de control. Los métodos poblacionales aportan diversidad. Por ejemplo, Abdullah et al. combinaron un mecanismo inspirado en electromagnetismo con Great Deluge [5]. Su coste suele ser una evaluación más cara y un número mayor de parámetros. La revisión de Abdipoor et al. confirma que la hibridación se ha convertido en una línea dominante, pero también señala la dificultad de comparar configuraciones y presupuestos heterogéneos [6].

I-B. Métodos híbridos

La frontera entre metaheurística y método híbrido no es rígida. Un algoritmo memético aplica mejora local a individuos generados por una población. Una matheurística destruye parte del horario y delega la reparación en programación matemática o programación por restricciones. Kiefer et al. emplean ALNS con una reparación exacta basada en CPLEX [7]. El modelo resuelve con precisión el subproblema liberado y la búsqueda adaptativa decide qué parte conviene reconstruir. Esta combinación produce soluciones fuertes bajo los límites de ITC-2007, pero depende de un optimizador externo y de la escala del vecindario reparado.

La utilidad práctica del híbrido está en repartir funciones. El componente exacto protege la factibilidad o explora un subespacio bien delimitado. La metaheurística mantiene la diversidad y recorre regiones alejadas. A cambio, aumenta la dificultad de atribuir las mejoras a un componente y de repetir el experimento cuando faltan licencias, código o parámetros.

I-C. Aprendizaje automático

El aprendizaje automático se usa sobre todo para decidir cómo resolver una instancia, no para construir por sí solo el horario final. De Coster et al. describen cada instancia mediante 256 características, delimitan regiones del espacio de problemas y entrenan selectores capaces de recomendar un algoritmo [8]. El resultado es coherente con la ausencia

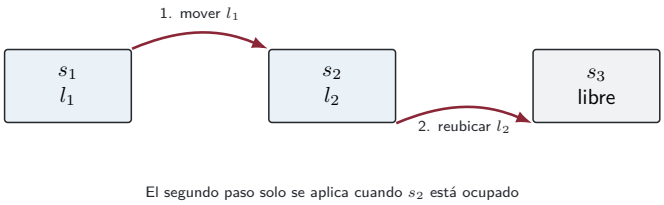


Figura 1. Movimiento LectureKick. La clase l_1 pasa de s_1 a s_2. Si s_2 contiene l_2, esta clase se desplaza a un hueco libre s_3. El movimiento se descarta cuando alguna reasignación viola disponibilidad, conflictos o unicidad de clase por periodo.

de un ganador universal. Una técnica puede dominar en instancias densas y perder su ventaja cuando cambian el número de currículos, la ocupación o la disponibilidad.

También se emplean modelos para predecir rendimiento, configurar parámetros y elegir operadores durante la búsqueda. Conviene distinguir estos usos de la configuración automática por carreras estadísticas. Ceschia et al. ajustan el algoritmo con `irace`, pero LectureKick sigue siendo una búsqueda local diseñada de forma explícita [9]. La ganancia procede de examinar sesgos y vecindarios, no de una red neuronal que aprenda a generar horarios.

II. LectureKick

La solución se representa mediante un vector con una entrada por clase. Cada entrada contiene un hueco formado por periodo y aula. La construcción inicial tiene tres etapas. Primero se asignan clases al azar respetando disponibilidad y ocupación. Después un Hill Climbing elimina los conflictos entre cursos. Una segunda fase de Hill Climbing incorpora las cuatro penalizaciones blandas y entrega al recocido simulado una solución factible de calidad razonable [9].

LectureKick extiende el movimiento LectureMove de Müller. Se selecciona una clase l_1 situada en s_1 y se propone el destino s_2. Si s_2 está libre, el movimiento termina con la reasignación de l_1. Si contiene una clase l_2, esta se traslada a un tercer hueco libre s_3. LectureMove solo permitía enviar l_2 al hueco liberado por l_1. La nueva libertad conecta más soluciones sin tener que introducir estados infactibles.

Las precondiciones comprueban que los destinos sean distintos, que cada curso tenga como máximo una clase en el periodo elegido y que profesores y currículos permanezcan sin conflictos. También verifican la disponibilidad y la libertad de s_3. Por tanto, la fase principal del algoritmo explora únicamente horarios factibles.

La selección no es uniforme. Un sesgo mueve clases de cursos que incumplen el mínimo de días hacia jornadas todavía vacías. Otro prioriza cursos presentes en muchos currículos para actuar sobre clases aisladas. El sesgo de estabilidad intenta reutilizar aulas ya asignadas al curso. La capacidad se trata de forma preventiva mediante aulas suficientes siempre que existan. Un último parámetro favorece que l_2 ocupe s_1, reproduciendo el intercambio del operador original. En la compilación `ENHANCED` el sesgo reactivo de capacidad queda desactivado y la preferencia por aulas adecuadas se aplica siempre que sea posible [10].

El recocido utiliza temperatura inicial 5.394, temperatura mínima 0.069 y factor de enfriamiento 0.988. En cada nivel corta el muestreo cuando la proporción de movimientos aceptados alcanza 0.133. Los sesgos finales son 0.202 para días mínimos, 0.122 para cursos críticos, 0.146 para estabilidad y 0.803 para intercambios. El umbral de curso crítico es tres.

III. Protocolo de reproducción

Se fijó el commit `9cb3eb6` del repositorio de Ceschia et al. y el commit `b40b14c` de EasyLocal++ 3. La compilación usa GNU G++ 16.2, C++17, optimización `-O3`, Boost.Program_options 1.92 y la macro `ENHANCED`. La campaña se ejecutó con MSYS2 UCRT64 sobre Windows 10. El artículo emplea Ubuntu 24.04 y GNU G++ 13.3. Esta diferencia impide comparar los tiempos de forma directa, aunque conserva el código, los parámetros y el número de iteraciones.

El estudio incluye las 19 instancias disponibles en el directorio `instances/comp`. Se omiten `comp11`, cuyo coste cero no aporta discriminación, y `comp15`, equivalente a `comp03` bajo UD2. Cada instancia se ejecuta con las semillas 0, 1 y 2. El método es `BHC-KSA` y el límite `virtual_timeout=192` convierte el tiempo de referencia en un número de iteraciones específico para cada instancia. El diseño suma 57 ejecuciones secuenciales ya completadas.

Antes de iniciar la campaña se ejecutó dos veces `comp01` con semilla 0 y un límite reducido. Ambas pruebas produjeron coste 5, cero violaciones y el mismo horario. Cada ejecución definitiva conserva el horario, la salida estándar, el código de retorno, el coste, las violaciones, el tiempo declarado por el solver y el tiempo de pared. El archivo `results.csv` contiene una fila por pareja instancia y semilla. Una fila se acepta solo si el proceso termina correctamente, el coste puede analizarse y las violaciones duras son cero. Con tres semillas no se pretende reproducir la distribución de 30 repeticiones publicada. El objetivo es comprobar el artefacto y proporcionar una muestra trazable para la comparación posterior.

Referencias

[1] S. Ceschia, L. D. Gaspero, and A. Schaerf, "Educational timetabling: Problems, benchmarks, and state-of-the-art results," *European Journal of Operational Research*, vol. 308, no. 1, pp. 1–18, 2023.

[2] T. Müller, "ITC2007 solver description: A hybrid approach," *Annals of Operations Research*, vol. 172, no. 1, pp. 429–446, 2009.

[3] R. Bellio, S. Ceschia, L. D. Gaspero, A. Schaerf, and T. Urli, "Feature-based tuning of simulated annealing applied to the curriculum-based course timetabling problem," *Computers & Operations Research*, vol. 65, pp. 83–92, 2016.

[4] T. Song, M. Chen, Y. Xu, D. Wang, X. Song, and X. Tang, "Competition-guided multi-neighborhood local search algorithm for the university course timetabling problem," *Applied Soft Computing*, vol. 110, p. 107624, 2021.

Cuadro I
Comparación crítica de las familias revisadas

Familia	Calidad esperada	Coste temporal	Parámetros	Generalización
Metaheurísticas	Alta con vecindarios y ajuste adecuados	Permiten devolver una solución en cualquier momento	Temperatura, enfriamiento, sesgos y reinicios	Sensibles a la estructura de la instancia
Híbridos	Muy alta cuando la reparación exacta cubre un subproblema útil	Mayor coste por movimiento y posible dependencia de licencias	Tamaño de destrucción, reparación y control de la búsqueda	Buena si el modelo exacto conserva la formulación
Aprendizaje automático	Indirecta, depende de los algoritmos candidatos	Añade entrenamiento, pero reduce decisiones durante la resolución	Características, modelo y conjunto de entrenamiento	Limitada fuera del espacio de instancias observado

Cuadro II
Configuración reproducida de BHC-KSA

Parámetro	Valor	Componente	Función
`mwd_bias`	0.202	LectureKick	Favorece días todavía no usados por el curso
`critical_course_bias`	0.122	LectureKick	Prioriza cursos presentes en varios currículos
`critical_course_threshold`	3	LectureKick	Define cuándo un curso se considera crítico
`stability_bias`	0.146	LectureKick	Favorece aulas ya utilizadas por el curso
`swap_bias`	0.803	LectureKick	Favorece que la clase desplazada ocupe s_1
`start_temperature`	5.394	Recocido	Controla la aceptación inicial de empeoramientos
`min_temperature`	0.069	Recocido	Establece el umbral de parada térmica
`cooling_rate`	0.988	Recocido	Reduce la temperatura después de cada nivel
`neighbors_accepted_ratio`	0.133	Recocido	Corta el nivel al alcanzar la proporción aceptada

[5] S. Abdullah, H. Turabieh, B. McCollum, and P. McMullan, "A hybrid metaheuristic approach to the university course timetabling problem," *Journal of Heuristics*, vol. 18, no. 1, pp. 1–23, 2012.

[6] S. Abdipoor, R. Yaakob, S. L. Goh, and S. Abdullah, "Meta-heuristic approaches for the university course timetabling problem," *Intelligent Systems with Applications*, vol. 19, p. 200253, 2023.

[7] A. Kiefer, R. F. Hartl, and A. Schnell, "Adaptive large neighborhood search for the curriculum-based course timetabling problem," *Annals of Operations Research*, vol. 252, no. 2, pp. 255–282, 2017.

[8] A. D. Coster, N. Musliu, A. Schaerf, J. Schoisswohl, and K. Smith-Miles, "Algorithm selection and instance space analysis for curriculum-based course timetabling," *Journal of Scheduling*, vol. 25, no. 1, pp. 35–58, 2022.

[9] S. Ceschia, F. D. Ros, L. D. Gaspero, and A. Schaerf, "Revisiting local search for curriculum-based course timetabling," *Computers & Operations Research*, vol. 195, p. 107621, 2026.

[10] IOLab, University of Udine, "Revisiting local search for curriculum-based course timetabling: Software and instances," GitHub repository, 2026, commit 9cb3eb63040241fd1954ba49abcaba64461702f6. [Online]. Available: https://github.com/iolab-uniud/cb-ctt